
Modelos basados en árboles

Aprendizaje de Máquinas, MDS (Primavera 2025)

Patricio Espinoza A.^{* 1} Francisco Vásquez L.^{* 1 2} Álvaro Márquez² Víctor Hernández M.³

Abstract

Este documento contiene la resolución de los problemas de la Tarea 6 del ramo Aprendizaje de Máquinas del Máster en Ciencia de Datos de la Universidad de Chile.

1. Preliminar

Parte preliminar

El conjunto de datos consiste en características de las células presentes en imágenes que permiten detectar el cáncer de mama.

La variable objetivo es 'diagnosis', que indica si el tumor es Maligno o Benigno.

A partir de las imágenes, hay diez características principales calculadas para cada núcleo celular:

1. **radius:** media de las distancias desde el centro del núcleo hasta los puntos de su perímetro.
2. **texture:** desviación estándar de los valores de escala de grises (variabilidad de la textura).
3. **perimeter:** longitud del perímetro del núcleo celular.
4. **area:** área del núcleo celular.
5. **smoothness:** variación local en la longitud del radio (grado de suavidad de los bordes).
6. **compactness:** relación entre el perímetro y el área ($\text{perímetro}^2 / \text{área} - 1.0$).
7. **concavity:** grado de concavidad de las partes hundidas del contorno.
8. **concave points:** número de puntos o segmentos cóncavos del contorno.
9. **symmetry:** simetría del núcleo celular.
10. **fractal_dimension:** medida de la complejidad del contorno ("aproximación de línea costera" - 1).

Para cada una de estas diez características se calcularon tres tipos de valores:

- **_mean:** valor promedio de la característica.
- **_se:** error estándar del valor medio (variabilidad interna en las mediciones).
- **_worst:** promedio de los tres valores más altos registrados para esa característica.

Resultando en las variables presentes en la Tabla 1, para las cuales podemos ver que no hay nulos, y que la mayoría de datos son 'float64'.

En cuanto a la distribución de la variable objetivo 'diagnosis', a partir de la Tabla 2 se aprecia que predomina la Clase Benigna (357 casos o 62.75%) por sobre la Maligna (212 casos o 37.25%).

De la Figura 1 podemos ver que para varias características, como por ejemplo las distribuciones de radius_mean, concavity_mean, , texture_worst, perimeter_worst y concave_points_worst entre otras más, difieren claramente dependiendo de si se trata de un tumor Benigno o Maligno, lo que a priori nos permite identificar variables del dataset que deberían ser las más influyentes.

Al complementar esta observación con las estadísticas descriptivas, podemos ver que para aquellas variables 'mean' (Tablas 3 y 4) cuando es Maligno el promedio de valores de perimeter_mean y area_mean son mucho mayores a cuando es Benigno.

De igual forma, para aquellas 'se' (Tablas 5 y 6) se observa que el promedio y desviación estándar de perimeter_se y area_se es mucho mayor cuando el diagnóstico es Maligno que cuando es Benigno.

Finalmente, para las variables 'worst' (Tablas 7 y 8) destacan nuevamente perimeter y area. En este caso también es mucho más claro que influye también concavity y concave, la cual también tuvo mayores valores en tablas anteriores para el caso Maligno.

Parte (a)

Los árboles manejan de forma natural las variables dummies binarias (0/1), ya que las divisiones en cada nodo son de tipo “ $\leq$ umbral”, y para una dummy la separación óptima siempre ocurre entre 0 y 1. Por ello, no requieren ningún tratamiento adicional.

En cambio, para variables categóricas con más de dos niveles es preferible usar one-hot encoding, de modo que cada categoría se represente como una dummy independiente y el árbol pueda evaluarlas mediante divisiones binarias coherentes.

Basandonos en la Tabla 9 se seleccionaron las 3 variables con varianza más alta (area_worst, area_mean y area_se) ya que pueden aportar la mayor información y se categorizaron en bajo, medio y alto.

Parte (b)

A partir de las Figuras 2, 3 y 4, junto con las Tablas 10, 11 y 12, podemos observar que el radio tiene una alta correlación con el perímetro y el área, al igual que entre el perímetro y el área entre sí. Esto mismo se repite para compactness y concavity, así como entre concavity y concave points.

En los modelos basados en árboles, la multicolinealidad no afecta significativamente el rendimiento, porque en cada nodo el árbol selecciona solo una de las variables correlacionadas para el split. Sin embargo, sí puede perjudicar la interpretabilidad ya que las variables altamente correlacionadas tienden a repartirse la importancia, diluyendo su aporte individual.

En clusterización, la alta correlación genera redundancia en el espacio de características, lo que puede sesgar métodos basados en distancias como K-means, sobreponderando ciertas direcciones y dando lugar a clusters menos estables o menos representativos.

2. Clusterización**Parte (c)**

De la Figura 5 podemos ver que al utilizar PCA con 2 componentes se realiza una separación visual entre las clases Maligna y Beninga, con una pequeña superposición en la frontera. Esto indica que una parte importante de la información discriminante del dataset queda capturada por estas dos componentes.

PC1 y PC2 capturan una buena parte de la estructura discriminante al alcanzar cerca del 63% de la varianza. Por tanto, usar el dataset proyectado por PCA puede ser útil porque las PCs son ortogonales y concentran información relevante, reduciendo la redundancia del dataset (dimensionalidad y multicolinealidad).

Esto puede beneficiar métodos de clusterización o modelos sensibles a la dimensionalidad. Y su principal desventaja es la pérdida de interpretabilidad, ya que las componentes no corresponden a variables originales, además de la pérdida de información correspondiente al 37% restante de la varianza.

Parte (d)

A partir de la Figura 6 y Tabla 13 podemos ver que a través del método del codo las mayores reducciones de SSE ocurren para $k = 2 \rightarrow k = 3$ y $k = 3 \rightarrow k = 4$. Luego, a través del Coeficiente de Silhouette podemos ver que el máximo ocurre para $k = 2$ (*Silhouette* = 0.3434). En base a esto, el mejor Cluster es aquel con $k = 2$.

Para este $k = 2$ se obtuvo un *ARI* = 0.6536, indicando una similitud moderadamente alta respecto a los datos originales. La cantidad de clusters encontrados coincide también con la cantidad de etiquetas (Beninga y Maligna), mostrando que la estructura no supervisada es capaz de capturar correctamente la estructura real. Sin embargo, de la Figura 7 podemos ver que K-Means tiene problemas para clasificar correctamente aquellos datos que se solapan en la frontera de los clusters (Comparar con Figura 5).

Parte (e)

DBSCAN es un método de clustering que agrupa los puntos que forman regiones densas y marca como ruido a los puntos aislados.

Tiene dos hiperparámetros principales:

- *eps* (ϵ): El radio del vecindario alrededor de cada punto.
- *min_samples*: Cantidad mínima de puntos dentro del vecindario para que un punto sea considerado núcleo.

DBSCAN clasifica los puntos como:

- Núcleo: Tienen $\geq \text{min_samples}$ en su radio ϵ , formando un cluster.
- Borde: Están cerca de un núcleo, pero no alcanzan *min_samples*.
- Ruido: No están en ninguna región densa.

Al aplicar el algoritmo con *min_samples* = 5 y distintos *eps* se obtuvieron los resultados de la Tabla 14 en donde para todos los *eps* se obtuvo más del 50% de datos clasificados como ruido. Para $1.5 \leq \text{eps} \leq 2.0$ encuentra clusters con mucho ruido, *silhouette* negativo indicando mala estructura y *ARI* ≈ 0 que implica que los resultados no se parecen a la clasificación real. Esto se puede visualizar en la Figura 8 donde los clusters difieren mucho de los datos reales (Cluster -1 es ruido).

Esto contrasta con K-means, que con $k=2$ obtuvo un *ARI* ≈ 0.65 y una separación visual clara en el espacio PCA. La diferencia se debe a la naturaleza de los algoritmos ya que

K-means asume clusters compactos y bien separados por la distancia euclidiana, lo cual coincide razonablemente con la estructura del dataset. En cambio, DBSCAN requiere regiones densas y bien delimitadas en el espacio de características, algo que se vuelve difícil de detectar en alta dimensionalidad y cuando las clases presentan densidades diferentes. Debido a esto DBSCAN interpreta la mayor parte del dataset como ruido y no logra capturar la separación entre benignos y malignos.

Parte (f)

Mediante el dendrograma de la Figura 9 podemos determinar que $k = 2$ es un buen punto de corte para diferenciar los datos, lo cual es complementado con los Coeficientes de Silhouette ($Silhouette_{k_2} = 0.3394$ y $ARI(ARI_{k_2} = 0.5750)$ de la Tabla 15. Al graficar los clusters obtenidos sobre el espacio PCA podemos ver que se mantiene gran parte de la estructura original de los datos para ambas clases (Figura 10).

Al comparar K-Means, DBSCAN y Clustering Jerárquico Aglomerativo para $k = 2$ podemos ver que el ARI más alto fue el de K-Means con 0.6536, al igual que el Coeficiente de Silhouette con 0.3434. Sin embargo, el Clustering Aglomerativo obtuvo resultados similares y una buena distinción al proyectar sobre PCA, lo cual difiere mucho del mal rendimiento de DBSCAN, que tuvo un ARI cercano a 0 y Coeficiente de Silhouette negativo.

3. Modelos basados en árboles

Parte (g)

Al hacer un split estratificado de 80% entrenamiento y 20% prueba se obtienen distribuciones del 62% para la clase Benigna y del 37% para la clase Maligna (Tabla 16).

Parte (h)

El Decision Tree Classifier tiene los siguientes parámetros:

- Criterion: Mide la calidad de una división en cada nodo del árbol (Gini o Entropía), permitiendo al árbol seleccionar aquellas que mejor particionen a las clases.
- min_samples_leaf: Establece el mínimo de muestras por hoja, permite evitar hojas pequeñas para reducir el sobreajuste.
- max_depth: Limita la profundidad del árbol para que aprenda solo las divisiones más importantes, controlando así la complejidad y previniendo el sobreajuste.
- random_state: Fija la semilla aleatoria para garantizar reproducibilidad del modelo y para decisiones propias del modelo.

De la Tabla 17 podemos ver que en el conjunto de entre-

namiento se obtuvieron 0 FP y FN, además de métricas de 1.0 en Accuracy, Precision, Recall y F1-Score. Para el conjunto de Prueba se obtuvieron 4 FP y 4 FN, disminuyendo levemente cada una de las métricas. Esto indica un claro sobre ajuste del modelo a los datos de entrenamiento, con una pérdida de generalización debido al uso de los parámetros por defecto.

Finalmente, al compararlo el mismo modelo con los mismos datos pero con un Random State distinto (Tabla 18) podemos ver que ocurrió una pequeña variación en los Falsos Positivos y Verdaderos Negativos, generando métricas ligeramente mejores. Esto sucedió ya que el Random State influye en las decisiones propias del modelo, en particular para el desempate entre atributos con la misma ganancia y la selección de características en árboles aleatorios.

Parte (i)

De la Figura 11 se observa que la curva ROC asciende rápidamente hacia la esquina superior izquierda, manteniéndose claramente por sobre la diagonal aleatoria. Esto indica que el modelo discrimina mejor que un clasificador no informativo. Además, el valor obtenido de $AUC = 0.925$ sugiere un rendimiento elevado, dado que el área bajo la curva se acerca a 1, lo que implica una buena capacidad para distinguir entre ambas clases.

Parte (j)

Al aplicar GridSearch con la grilla de hiperparámetros:

- criterion: gini, entropy, log_loss
- min_samples_leaf: 1, 2, 10,
- max_depth: 2, 5, 15

Se obtuvo que el mejor modelo (criterion = Gini, max_depth = 5 y min_samples_leaf = 10) alcanzó un AUC de 0.9929 en Entrenamiento y 0.9688 en Prueba. Asimismo, alcanzó una Accuracy de 0.9582 y 0.9123, y un F1-Score de 0.9422 y 0.8649, respectivamente. Estos resultados muestran un buen rendimiento y una adecuada capacidad de generalización, sin evidencias de sobreajuste.

Parte (k)

A partir de la Figura 12 podemos ver que las variables más importantes son perimeter_worst, concave points_worst, texture_worst, area_se, concave points_mean y fractal_dimension_se.

En particular destaca como variables señaladas como candidatas para ser categorizadas por su varianza se repiten (Tabla 9) como lo es perimeter_worst. Esto indica que en caso de usarse categóricas el rendimiento debería ser similar.

Nota: No se incluyó la variable categórica creada ni el dataset con PCA

Parte (l)

Al incorporar la variable proveniente de K-Means, los resultados de la Tabla 20 muestran una mejora de 0.0047 en el F1-Score promedio de CV. En un contexto médico esta ganancia puede justificar su uso, aunque en otros escenarios podría no ser conveniente, dado que añadir un paso previo de K-Means implica un mayor costo computacional, especialmente en conjuntos de datos grandes.

4. Modelos basados en árboles, profundización**Parte (m)**

La poda por Minimal Cost Complexity consiste en reducir el tamaño del árbol eliminando ramas que aportan poco a la capacidad predictiva. Para ello se asigna una función de costo que penaliza la complejidad del árbol:

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

- $R(T)$ es el error de entrenamiento del árbol.
- $|T|$ es el número de hojas.
- α es el parámetro de complejidad.

El proceso genera una secuencia de árboles podados, desde el más grande (sin podar) hasta el más simple, eliminando en cada paso la rama cuya poda aumenta menos el costo penalizado.

El parámetro `ccp_alpha` controla la magnitud de esta penalización:

- Si `ccp_alpha = 0`, no se penaliza la complejidad y el árbol puede crecer sin límite.
- Si `ccp_alpha` aumenta, la penalización favorece árboles más pequeños y generalizables.

Parte (n)

A partir de la Figura 13 se puede ver el cambio de la Profundidad del árbol y de la Cantidad de nodos de acuerdo al `cc_alpha` usado. En particular, a medida que este aumenta ambos tienden a disminuir rápidamente debido a que el parámetro implica una mayor penalización y poda. Por otro lado, de la Figura 14 se puede ver que la Accuracy disminuye mientras el `cc_alpha` aumenta, tanto en Entrenamiento como en Prueba. Esto ocurre ya que debido a la poda, el árbol en entrenamiento pierde la capacidad de ajustarse bien a los datos vistos, mientras que en Prueba ocurre que debido a una poda muy agresiva, se sobrepasa el umbral de añadir un poco de sesgo mediante la poda para que mejore en Prueba, y en su lugar se vuelve incapaz de generalizar.

El mejor valor obtenido validación fue para `cc_alpha = 0.0021`, ya que aunque disminuye la accuracy de entrenamiento, la poda generada por este valor permite una mejor

capacidad de generalización frente a datos nuevos.

Finalmente, de las Figuras 15 y 16 se puede observar que al usar el parámetro óptimo de `cc_alpha` el árbol efectivamente disminuye su profundidad y cantidad de nodos, lo cual es lo que le permite una mejor capacidad de generalización.

Parte (ñ)

A partir de la Tabla 21 podemos ver que tanto para un Random Forest como para un Decision Tree que implementa Bagging se obtienen la misma Accuracy para Entrenamiento (1.0) y Prueba (0.9737). Esto indica que ambos modelos fueron capaces de generalizar bien pese a su Accuracy perfecta en Entrenamiento.

Las diferencias entre un Bagging y un Random Forest radican en como introducen aleatoriedad para la reducción de la varianza. En Bagging todos los árboles se entrenan sobre diferentes muestras bootstrap del conjunto de datos, pero cada árbol puede utilizar todas las variables en cada división. En cambio, en Random Forest, además del muestreo bootstrap, en cada nodo se considera solo un subconjunto aleatorio de variables para realizar el split, lo que aumenta la diversidad entre los árboles y reduce aún más la correlación entre ellos.

En este caso ambas técnicas obtienen resultados idénticos debido a que el conjunto de datos tiene una estructura clara y altamente separable, por lo que la aleatoriedad adicional de Random Forest no aporta una mejora significativa sobre el Bagging clásico.

Parte (o)

La interpretabilidad de Random Forest puede analizarse a través de la estimación de la importancia de las variables para las predicciones realizadas. Una forma es la importancia por impureza (Gini), que mide la reducción promedio de impureza que produce cada variable en los distintos árboles del bosque. En este caso, de la Tabla 22 podemos ver que las variables más relevantes fueron `concave points_worst`, `concave points_mean`, `area_worst` y `radius_worst`.

Otra aproximación más robusta es la importancia por permutación, que evalúa la caída en el desempeño del modelo al permutar cada variable. Bajo este criterio, de la misma Tabla 22, podemos ver que destacan nuevamente `concave points_worst`, `concave points_mean`, `area_worst` y `radius_worst`, confirmando una consistencia en los resultados.

Parte (p)

El método de Boosting obtiene su mayor ganancia cuando los árboles son de baja profundidad y alto sesgo (modelos más débiles). De esta forma cada árbol corrige iterativamente los errores del conjunto anterior, permitiendo reducir progresivamente el sesgo del modelo global sin aumentar excesivamente la varianza.

Por otra parte, Bagging es más efectivo cuando los árboles base son modelos fuertes, es decir, árboles profundos con alta varianza ya que el promediado de múltiples modelos entrenados en distintos subconjuntos bootstrap estabilizará la predicción.

A partir de la Tabla 23 podemos corroborar como efectivamente una profundidad mayor en el árbol provoca una accuracy más baja en el conjunto de Prueba, cuando con $max_depth = 1$, ya se alcanzaba la máxima Accuracy en Prueba.

Parte (q)

De la Tabla 24 y Figura 17 podemos ver el desempeño de árboles que aplican Bagging, Random Forest y Gradient Boost para distintos estimadores K . En particular se puede apreciar que luego de $k = 25$ Bagging no mejora en su Accuracy en el conjunto de Prueba, mientras que Random Forest tiene un comportamiento intercalado, subiendo para $K = 50$, bajando en $K = 75$ y subiendo nuevamente para $K = 100$, lo cual se puede explicar debido a la aleatoriedad introducida por el Bootstrap de los datos y la selección aleatoria de variables en cada split.

Por otra parte, de Gradient Boost vemos como el aumentar los estimadores implica una mejora continua en el intervalo $K \in [10, 75]$, donde luego de este pasa a estancarse y dejar de mejorar.

Comparando los mejores resultados, tanto Bagging como Random Forest alcanzan la mayor Accuracy en Prueba (0.9736). Sin embargo, Random Forest sería el mejor modelo debido a que iguala el mejor rendimiento y es más robusto frente a variables correlacionadas como lo son nuestros datos, además de que permite un análisis más completo sobre la importancia de las variables.

5. Generalización

Parte (r)

Para evaluar la generalización de los modelos se aplicó primero un escalado mediante RobustScaler y StandardScaler. Se evitó un posible Data Leakage al escalar mediante la separación en los conjuntos X_{train} y X_{test} , para luego aplicar de forma separada los respectivos escalados.

Parte (s)

Al utilizar un modelo de Gradient Boosting con $max_depth = 1$ y $n_estimators = 75$ (basado en resultados anteriores) sobre los conjuntos de datos originales, y aquellos que fueron escalados mediante RobustScaler y StandardScaler se obtuvieron los resultados de la Tabla 25. Se puede observar que los tres modelos alcanzan un $F1 = 0.9791$ en Entrenamiento y un $F1 = 0.95$ en Prueba.

Parte (t)

De los resultados obtenidos se observó que independientemente del escalado usado se obtuvieron los mismos resultados que para el conjunto de datos original. Esto se debe a que los modelos basados en árboles no son sensibles al escalado de las variables puesto que las decisiones de partición dependen solo del orden relativo de los valores y no de su magnitud absoluta. Esto significa que el escalado no altera los puntos de corte óptimos ni la estructura de los árboles, por lo que el modelo aprendido es el mismo en los tres casos.

Debido a esto ocurre que, a diferencia de modelos basados en distancia o gradiente como SVM o regresión logística, el escalado no tiene impacto en el rendimiento de Gradient Boosting.

A. Anexo

A.1. Pregunta 1

A.1.1. PARTE PRELIMINAR

Table 1. Resumen de variables del conjunto de datos

N°	Variable	Valores no nulos	Tipo de dato
0	id	569	int64
1	diagnosis	569	object
2	radius_mean	569	float64
3	texture_mean	569	float64
4	perimeter_mean	569	float64
5	area_mean	569	float64
6	smoothness_mean	569	float64
7	compactness_mean	569	float64
8	concavity_mean	569	float64
9	concave points_mean	569	float64
10	symmetry_mean	569	float64
11	fractal_dimension_mean	569	float64
12	radius_se	569	float64
13	texture_se	569	float64
14	perimeter_se	569	float64
15	area_se	569	float64
16	smoothness_se	569	float64
17	compactness_se	569	float64
18	concavity_se	569	float64
19	concave points_se	569	float64
20	symmetry_se	569	float64
21	fractal_dimension_se	569	float64
22	radius_worst	569	float64
23	texture_worst	569	float64
24	perimeter_worst	569	float64
25	area_worst	569	float64
26	smoothness_worst	569	float64
27	compactness_worst	569	float64
28	concavity_worst	569	float64
29	concave points_worst	569	float64
30	symmetry_worst	569	float64
31	fractal_dimension_worst	569	float64

Table 2. Distribución de la variable objetivo *diagnosis*

Clase	Frecuencia	Porcentaje (%)
B (Benigno)	357	62.75
M (Maligno)	212	37.25
Total	569	100.00

Tarea 6, Aprendizaje de Maquinas. MDS, Universidad de Chile

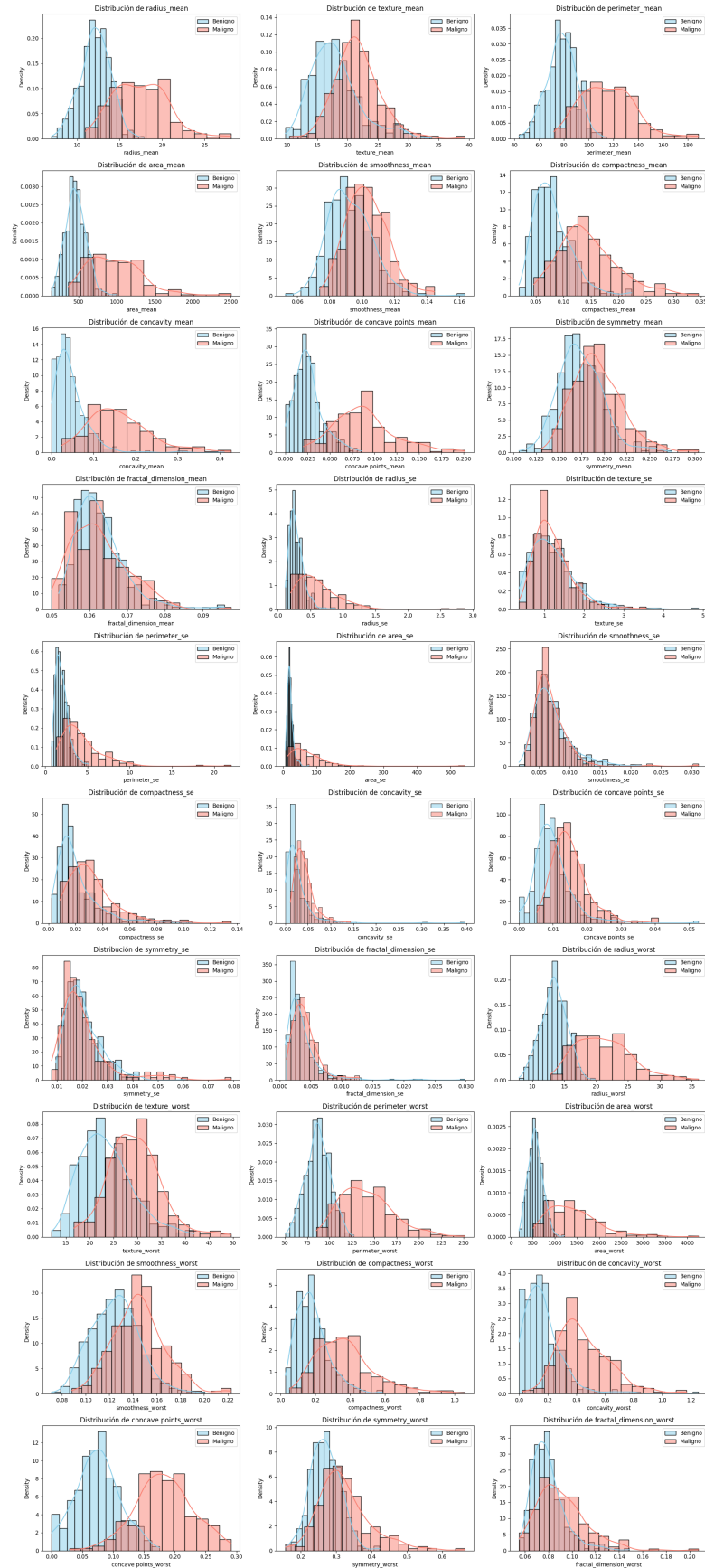


Figure 1. Distribución de las variables continuas de acuerdo al tipo de diagnóstico.

Table 3. Variables _mean — Clase **Benigna**

Variable	count	mean	std	min	25%	50%	75%	max
radius_mean	357	12.1	1.8	7.0	11.1	12.2	13.4	17.9
texture_mean	357	17.9	4.0	9.7	15.2	17.4	19.8	33.8
perimeter_mean	357	78.1	11.8	43.8	70.9	78.2	86.1	114.6
area_mean	357	462.8	134.3	143.5	378.2	458.4	551.1	992.1
smoothness_mean	357	0.09	0.01	0.05	0.08	0.09	0.10	0.16
compactness_mean	357	0.08	0.03	0.02	0.06	0.08	0.10	0.22
concavity_mean	357	0.05	0.04	0.00	0.02	0.04	0.06	0.41
concave points_mean	357	0.03	0.02	0.00	0.02	0.02	0.03	0.09
symmetry_mean	357	0.17	0.02	0.11	0.16	0.17	0.19	0.27
fractal dimension_mean	357	0.06	0.01	0.05	0.06	0.06	0.07	0.10

Table 4. Variables _mean — Clase **Maligna**

Variable	count	mean	std	min	25%	50%	75%	max
radius_mean	212	17.5	3.2	11.0	15.1	17.3	19.6	28.1
texture_mean	212	21.6	3.8	10.4	19.3	21.5	23.8	39.3
perimeter_mean	212	115.4	21.9	71.9	98.7	114.2	129.9	188.5
area_mean	212	978.4	367.9	361.6	705.3	932.0	1203.8	2501.0
smoothness_mean	212	0.10	0.01	0.07	0.09	0.10	0.11	0.14
compactness_mean	212	0.15	0.05	0.05	0.11	0.13	0.17	0.35
concavity_mean	212	0.16	0.08	0.02	0.11	0.15	0.20	0.43
concave points_mean	212	0.09	0.03	0.02	0.06	0.09	0.10	0.20
symmetry_mean	212	0.19	0.03	0.13	0.17	0.19	0.21	0.30
fractal dimension_mean	212	0.06	0.01	0.05	0.06	0.06	0.07	0.10

Table 5. Variables _se — Clase **Benigna**

Variable	count	mean	std	min	25%	50%	75%	max
radius_se	357	0.284	0.113	0.112	0.207	0.258	0.342	0.881
texture_se	357	1.220	0.589	0.360	0.796	1.108	1.492	4.885
perimeter_se	357	2.000	0.771	0.757	1.445	1.851	2.388	5.118
area_se	357	21.135	8.843	6.802	15.260	19.630	25.030	77.110
smoothness_se	357	0.007	0.003	0.002	0.005	0.007	0.009	0.022
compactness_se	357	0.021	0.016	0.002	0.011	0.016	0.026	0.106
concavity_se	357	0.026	0.033	0.000	0.011	0.018	0.031	0.396
concave points_se	357	0.010	0.006	0.000	0.006	0.009	0.012	0.053
symmetry_se	357	0.021	0.007	0.010	0.016	0.019	0.024	0.061
fractal dimension_se	357	0.004	0.003	0.001	0.002	0.003	0.004	0.030

Table 6. Variables `_se` — Clase **Maligna**

Variable	count	mean	std	min	25%	50%	75%	max
radius_se	212	0.609	0.345	0.194	0.390	0.547	0.757	2.873
texture_se	212	1.211	0.483	0.362	0.893	1.103	1.429	3.568
perimeter_se	212	4.324	2.569	1.334	2.716	3.680	5.206	21.980
area_se	212	72.672	61.355	13.990	35.763	58.455	94.000	542.200
smoothness_se	212	0.007	0.003	0.003	0.005	0.006	0.008	0.031
compactness_se	212	0.032	0.018	0.008	0.020	0.029	0.039	0.135
concavity_se	212	0.042	0.022	0.011	0.027	0.037	0.050	0.144
concave points_se	212	0.015	0.006	0.005	0.011	0.014	0.017	0.041
symmetry_se	212	0.020	0.010	0.008	0.015	0.018	0.022	0.079
fractal dimension_se	212	0.004	0.002	0.001	0.003	0.004	0.005	0.013

Table 7. Estadísticas descriptivas de las variables `_worst` para la clase Benigna (B)

Variable	count	mean	std	min	25%	50%	75%	max
radius_worst	357	13.38	1.98	7.93	12.08	13.35	14.80	19.82
texture_worst	357	23.52	5.49	12.02	19.58	22.82	26.51	41.78
perimeter_worst	357	87.01	13.53	50.41	78.27	86.92	96.59	127.10
area_worst	357	558.90	163.60	185.20	447.10	547.40	670.00	1210.00
smoothness_worst	357	0.1250	0.0200	0.0712	0.1104	0.1254	0.1376	0.2006
compactness_worst	357	0.1827	0.0922	0.0273	0.1120	0.1698	0.2302	0.5849
concavity_worst	357	0.1662	0.1404	0.0000	0.0771	0.1412	0.2216	1.2520
concave points_worst	357	0.0744	0.0358	0.0000	0.0510	0.0743	0.0975	0.1750
symmetry_worst	357	0.2702	0.0417	0.1566	0.2406	0.2687	0.2983	0.4228
fractal dimension_worst	357	0.0794	0.0138	0.0552	0.0701	0.0771	0.0854	0.1486

Table 8. Estadísticas descriptivas de las variables `_worst` para la clase Maligna (M)

Variable	count	mean	std	min	25%	50%	75%	max
radius_worst	212	21.13	4.28	12.84	17.73	20.59	23.81	36.04
texture_worst	212	29.32	5.43	16.67	25.78	28.95	32.69	49.54
perimeter_worst	212	141.37	29.46	85.10	119.33	138.00	159.80	251.20
area_worst	212	1422.29	597.97	508.10	970.30	1303.00	1712.75	4254.00
smoothness_worst	212	0.1448	0.0219	0.0882	0.1305	0.1435	0.1560	0.2226
compactness_worst	212	0.3748	0.1704	0.0513	0.2445	0.3564	0.4479	1.0580
concavity_worst	212	0.4506	0.1815	0.0240	0.3264	0.4049	0.5562	1.1700
concave points_worst	212	0.1822	0.0463	0.0290	0.1528	0.1820	0.2107	0.2910
symmetry_worst	212	0.3235	0.0747	0.1565	0.2765	0.3103	0.3592	0.6638
fractal dimension_worst	212	0.0915	0.0216	0.0550	0.0763	0.0876	0.1026	0.2075

A.1.2. PARTE A

Variable	Varianza
area_worst	324167.39
area_mean	123843.55
area_se	2069.43
perimeter_worst	1129.13
perimeter_mean	590.44
texture_worst	37.78
radius_worst	23.36
texture_mean	18.50
radius_mean	12.42
perimeter_se	4.09

Table 9. Varianza de variables numéricas del dataset

A.1.3. PARTE B

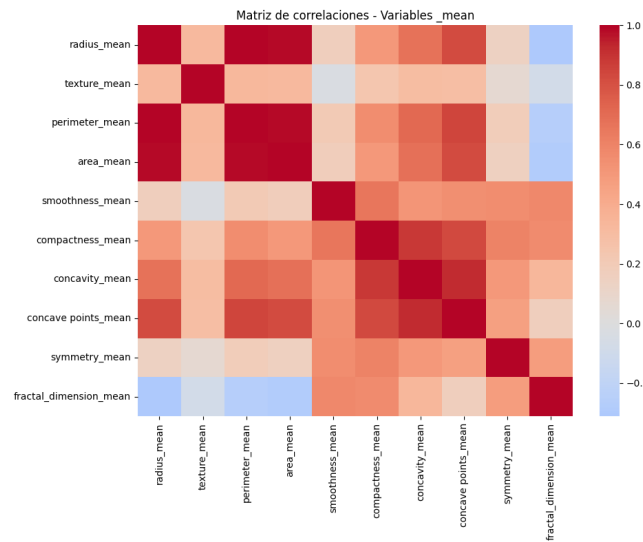


Figure 2. Correlación entre variables continuas, distinguidas por sufijo mean

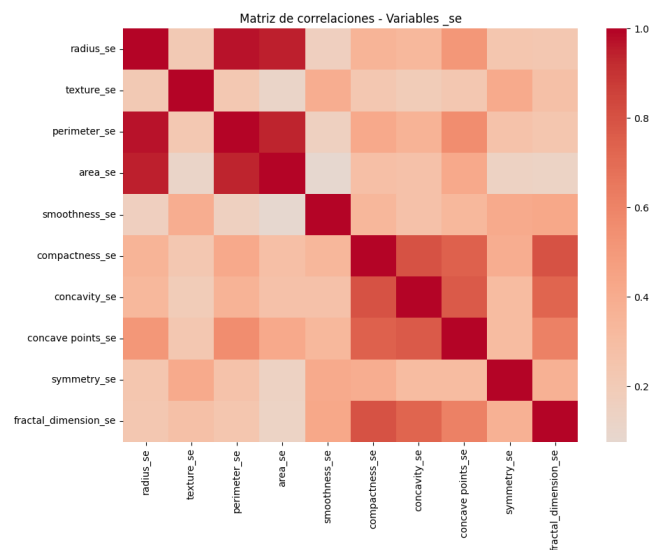


Figure 3. Correlación entre variables continuas, distinguidas por sufijo se.

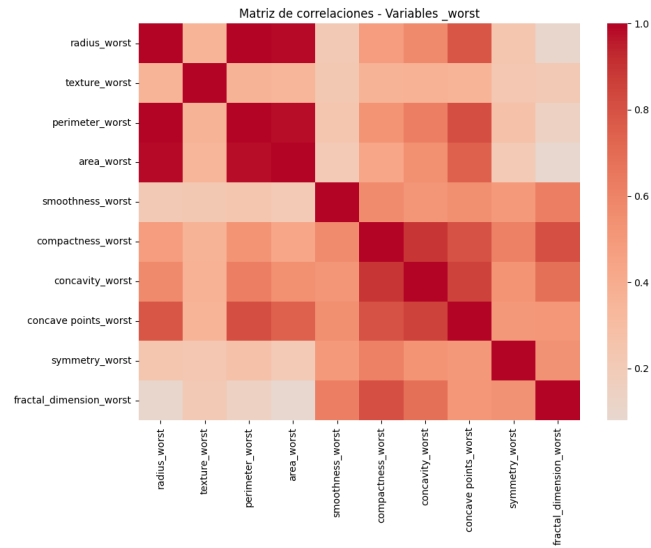


Figure 4. Correlación entre variables continuas, distinguidas por sufijo worst.

Table 10. Top correlaciones - _mean

Variable 1	Variable 2	Correlación
radius_mean	perimeter_mean	0.997855
radius_mean	area_mean	0.987357
perimeter_mean	area_mean	0.986507
concavity_mean	concave points_mean	0.921391
compactness_mean	concavity_mean	0.883121

Table 11. Top correlaciones - _se

Variable 1	Variable 2	Correlación
radius_se	perimeter_se	0.972794
radius_se	area_se	0.951830
perimeter_se	area_se	0.937655
compactness_se	fractal_dimension_se	0.803269
compactness_se	concavity_se	0.801268

Table 12. Top correlaciones - _worst

Variable 1	Variable 2	Correlación
radius_worst	perimeter_worst	0.993708
radius_worst	area_worst	0.984015
perimeter_worst	area_worst	0.977578
compactness_worst	concavity_worst	0.892261
concavity_worst	concave points_worst	0.855434

A.2. Clusterización

A.2.1. PARTE C

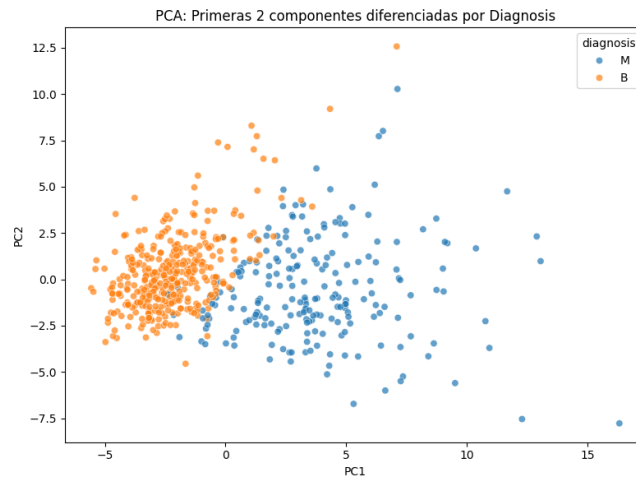


Figure 5. Proyección PCA: $Var(PC1) = 44, 27\%$ & $Var(PC2) = 18, 97\%$.

A.2.2. PARTE D

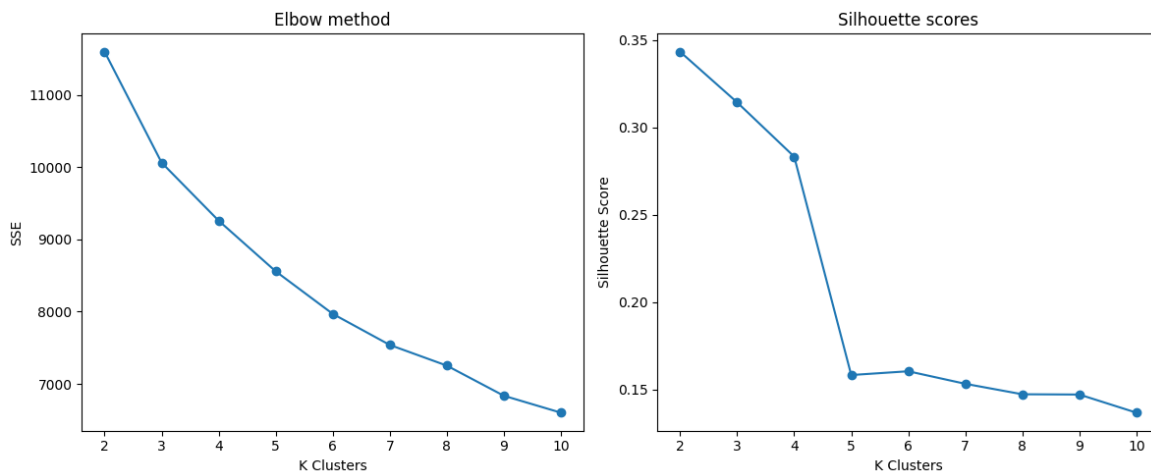


Figure 6. Método del codo y Silhouette Score para los datos originales escalados.

Table 13. K-Means en datos originales escalados: SSE y Silhouette por número de clusters

k	SSE	Silhouette
2	11595.53	0.3434
3	10061.80	0.3144
4	9258.99	0.2833
5	8558.66	0.1582
6	7970.26	0.1604
7	7540.32	0.1532
8	7254.33	0.1472
9	6837.63	0.1470
10	6603.40	0.1367

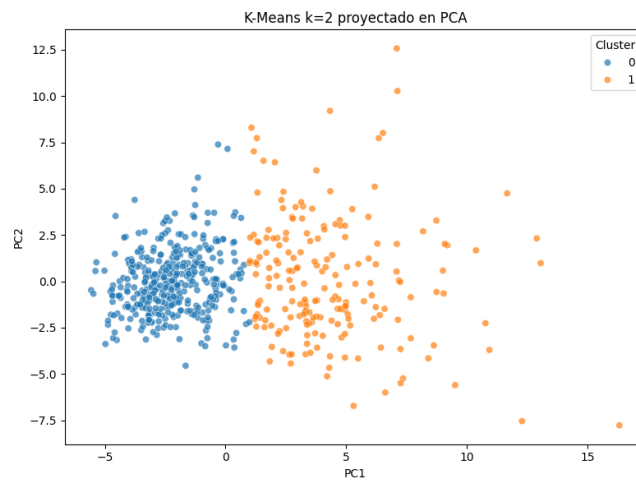
Table 13. Nota: $ARI_{k=2} = 0.6536$.

Figure 7. Clusters obtenidos por K-Means mediante las componentes de PCA.

A.2.3. PARTE E

Table 14. Resultados de DBSCAN con `min_samples = 5`

eps	Clusters	Noise %	Silhouette	ARI
0.5	0	100	nan	0.0000
1.0	0	100	nan	0.0000
1.5	1	96.66	nan	-0.0227
1.7	1	85.76	nan	-0.0343
1.8	2	78.56	-0.1445	0.0023
1.9	4	71.18	-0.2102	0.0534
2.0	4	65.20	-0.1985	0.0964

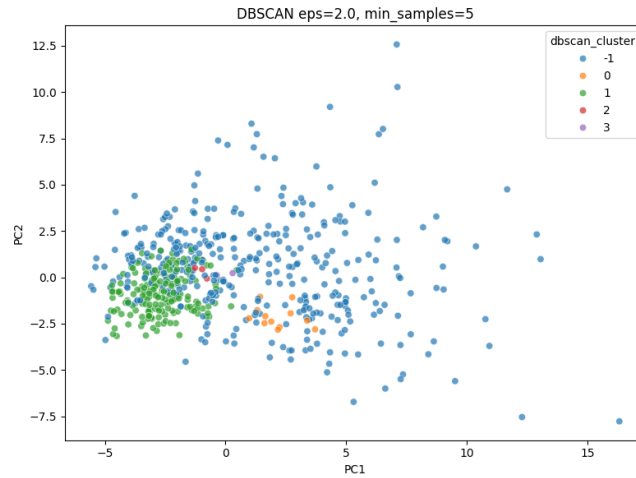


Figure 8. Visualización de DBSCAN en las Componentes Principales obtenidas mediante PCA.

A.2.4. PARTE F

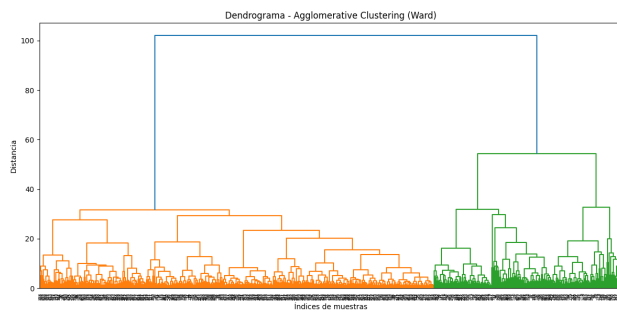


Figure 9. Dendrograma sobre los datos originales

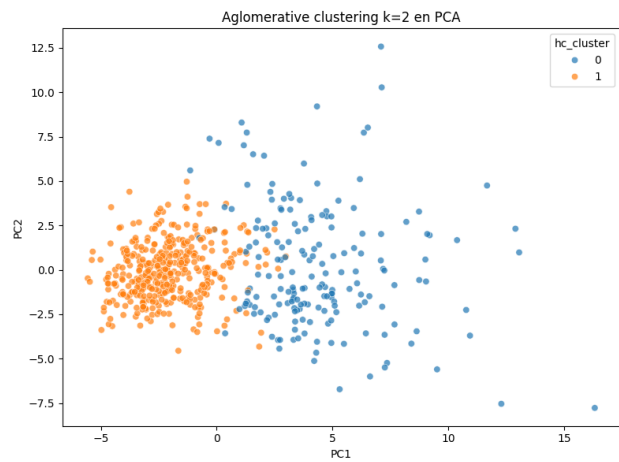


Figure 10. Clusters jerárquicos aglomerativos en las primeras dos componentes de PCA.

Table 15. Resultados de Agglomerative Clustering (datos originales)

k	ARI	Silhouette
2	0.5750	0.3394
3	0.5363	0.3301
4	0.5434	0.2982
5	0.5038	0.2434
6	0.2865	0.1199

A.3. Modelos Basados en Árboles

A.3.1. PARTE G

Table 16. Distribución de las clases con Split estratificado de entrenamiento y prueba (80/20)

Clase	Train (%)	Test (%)
0 (Benigna)	62.63	63.15
1 (Maligna)	37.36	36.84

A.3.2. PARTE H

Table 17. Resultados del DecisionTreeClassifier con parámetros por defecto

Dataset	TN	FP	FN	TP	Accuracy	Precision	Recall	F1-Score	Specificity	FPR	FNR
Train	285	0	0	170	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0000
Test	68	4	4	38	0.9298	0.9048	0.9048	0.9048	0.9444	0.0556	0.0952

Table 17. Nota: Random State=42.

Table 18. Resultados del DecisionTreeClassifier (Random State distinto)

Dataset	TN	FP	FN	TP	Accuracy	Precision	Recall	F1	Specificity	FPR	FNR
Train	285	0	0	170	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000	0.0000
Test	69	3	4	38	0.9386	0.9268	0.9048	0.9157	0.9583	0.0417	0.0952

Table 18. Nota: Random State=52.

A.3.3. PARTE I

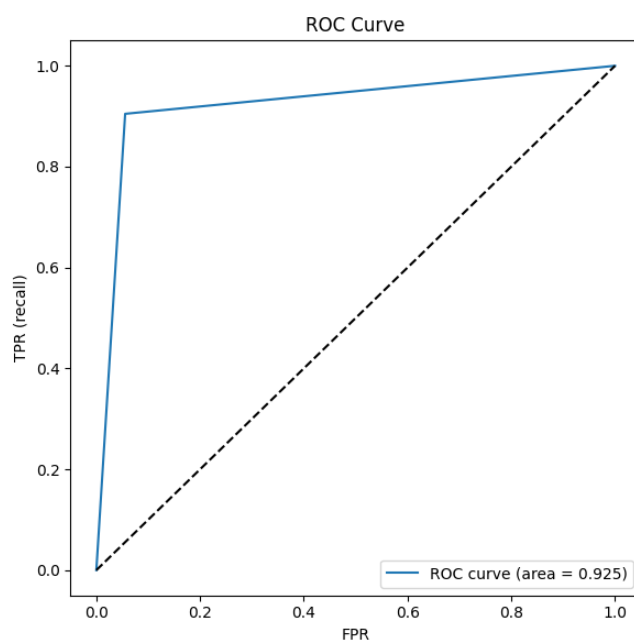


Figure 11. Curva ROC y métrica AUC del DecisionTreeClassifier con hiperparámetros por defecto.

A.3.4. PARTE J

Table 19. Resultados del DecisionTreeClassifier con GridSearch

Métrica	Train	Test
AUC	0.9929	0.9688
Accuracy	0.9582	0.9123
F1-Score	0.9422	0.8649
Mejores hiperparámetros: criterion=gini, max_depth=5, min_samples_leaf=10		

A.3.5. PARTE K

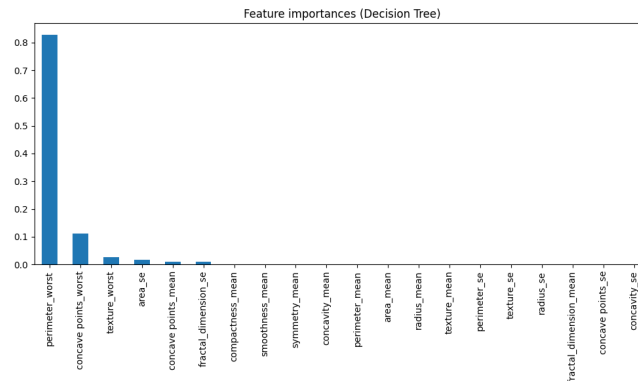


Figure 12. Importancia de las variables para el mejor modelo obtenido por GridSearch.

A.3.6. PARTE L

Table 20. Impacto de agregar labels de K-Means al DecisionTreeClassifier

Modelo	F1-Score CV (mean)
Sin K-Means	0.8929
Con K-Means	0.8976

A.3.7. PARTE N

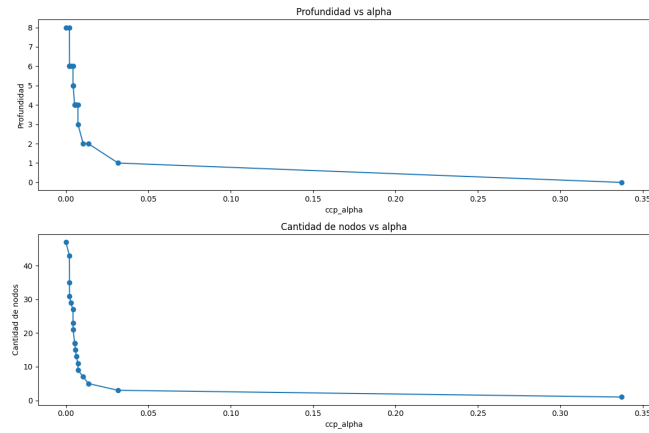


Figure 13. Variación de la profundidad y la cantidad de nodos según alpha.

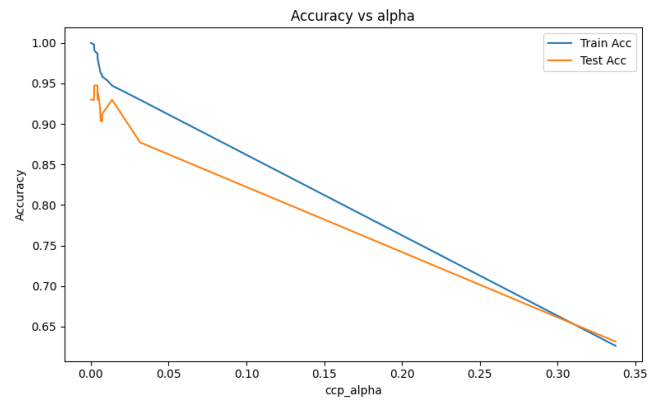


Figure 14. Variación de Accuracy vs alpha. Mejor Accuracy en Test para $\alpha = 0.0021$.

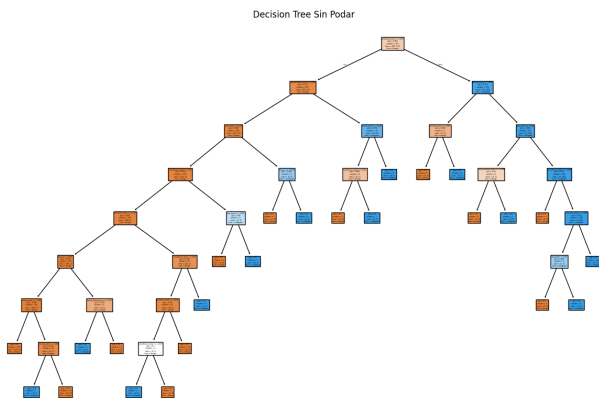


Figure 15. DecisionTree sin poda.

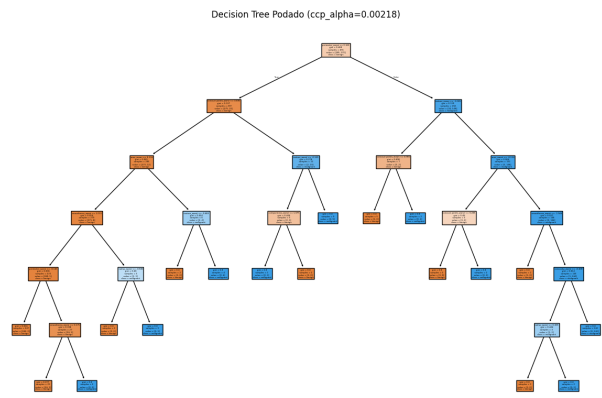


Figure 16. DecisionTree con poda ($cc_{\alpha} = 0.00218$).

A.3.8. PARTE Ñ

Table 21. Comparación de Accuracy entre DecisionTree podado y Bagging de Árboles

Dataset	Random Forest	Bagging de Árboles
Entrenamiento	1.0000	1.0000
Prueba	0.9737	0.9737

A.3.9. PARTE O

Table 22. Importancias de características en RandomForest: Gini vs Permutación

Variable	Gini	Permutación
concave points_worst	0.109992	0.034211
concave points_mean	0.104194	0.007018
area_worst	0.102906	0.016667
radius_worst	0.099437	0.015789
perimeter_mean	0.083051	0.007018
perimeter_worst	0.077482	0.015789
radius_mean	0.062782	0.006140
area_mean	0.048824	0.000000
concavity_mean	0.048072	
concavity_worst	0.046118	0.009649
area_se	0.039977	0.006140
radius_se	0.023394	0.008772
compactness_mean	0.019076	
compactness_worst	0.017881	0.003509
perimeter_se	0.015849	0.001754
texture_worst	0.014711	0.017544
texture_mean	0.012925	0.017544
smoothness_worst	0.011743	0.000877
symmetry_worst	0.009184	
fractal_dimension_se	0.006209	
fractal_dimension_mean		0.004386
texture_se		0.000000
symmetry_mean		0.000000
smoothness_mean		0.000000

A.3.10. PARTE P

Table 23. Modelo Gradient Boosting: Profundidad vs Accuracy

max_depth	Accuracy Entrenamiento	Accuracy Prueba
1	0.989011	0.964912
2	1.000000	0.964912
3	1.000000	0.964912
5	1.000000	0.929825
6	1.000000	0.921053
7	1.000000	0.929825
8	1.000000	0.929825
9	1.000000	0.929825
10	1.000000	0.929825

A.3.11. PARTE Q

Table 24. Desempeño de Bagging, Random Forest y Gradient Boosting para distintos valores de K

K	Bagging Test Acc	Random Forest Test Acc	GBoost Test Acc
10	0.964912	0.964912	0.921053
25	0.973684	0.964912	0.947368
50	0.973684	0.973684	0.956140
75	0.973684	0.964912	0.964912
100	0.973684	0.973684	0.964912
125	0.973684	0.964912	0.964912
150	0.973684	0.964912	0.964912
175	0.973684	0.964912	0.964912
200	0.973684	0.964912	0.964912

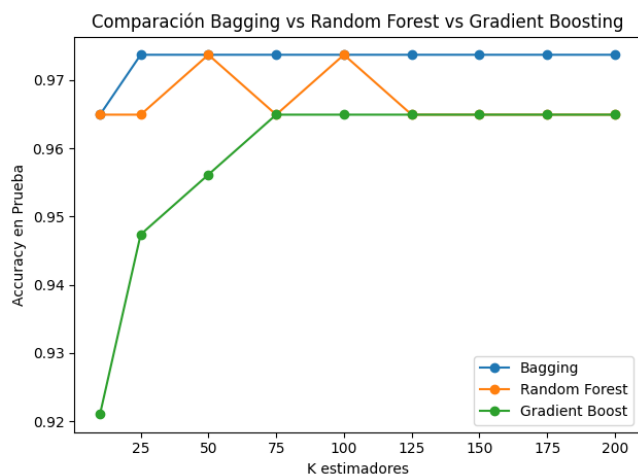


Figure 17. Accuracy en Prueba para modelos con distintos K estimadores.

A.4. Generalización

A.4.1. PARTE R

Table 25. F1-score en Entrenamiento y Prueba para distintos escaladores con Gradient Boosting

Modelo	F1 Entrenamiento	F1 Prueba
Original	0.9791	0.95
RobustScaler	0.9791	0.95
StandardScaler	0.9791	0.95